SIMATS ENGINEERING

ASSESSMENT TOOL 1 — ANALYTICAL PROBLEM SOLVING

Name	A.K. Kishore Kumar
Register No.	1913260079
Course Name	Artificial Intelligence
Course Code	CSA17

Submitted Answers

The following answers are prepared for the five questions in the supplied assessment sheet.

1. Water Jug Problem

Goal: Obtain exactly 2 gallons in the 4-gallon jug using only the 4-gallon and 3-gallon jugs.

State representation: (x, y), where x is the water in the 4-gallon jug and y is the water in the 3-gallon jug.

1. Fill the 3-gallon jug: (0, 3).

2. Pour the water from the 3-gallon jug into the 4-gallon jug: (3, 0).

3. Fill the 3-gallon jug again: (3, 3).

4. Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full. Only 1 gallon fits, so 2 gallons remain in the 3-gallon jug: (4, 2).

5. Empty the 4-gallon jug onto the ground: (0, 2).

6. Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug: (2, 0).

Result: The 4-gallon jug contains exactly 2 gallons.

2. Mars Rover as an Intelligent Agent

i. Percepts: Images from cameras; distance and obstacle readings; terrain and slope measurements; temperature and other environmental sensor readings; location and orientation; and scientific instrument readings from soil, rocks, and atmosphere.

ii. Operating environment: Mars is partially observable, uncertain, dynamic over time, and physically difficult to access. Communication with Earth has delays, energy and computing resources are limited, and terrain may contain hazards. The rover operates in a real-world, sequential environment.

iii. Actions: Move forward or backward; turn; stop; select a route; take photographs; use a robotic arm to collect or inspect samples; operate scientific instruments; store data; and send results to Earth.

iv. Performance evaluation: Measure safe distance travelled, successful arrival at target locations, quality and quantity of scientific data, accuracy of sample analysis, completion of mission goals, energy efficiency, equipment reliability, and avoidance of hazards.

v. Suitable architecture: A model-based, goal-based agent with utility-based decision-making is suitable. It maintains an internal map/state from sensor information, plans actions toward science goals, and compares routes or tasks based on safety, scientific value, time, and energy. A learning component can improve future decisions from collected experience.

3. Eight-Queens Problem: Search Space and Formulation

Problem: Place eight queens on an 8 × 8 chessboard so that no two queens attack each other. Therefore, no two queens may share a row, column, or diagonal.

State: A partial or complete placement of queens on the board. A convenient representation is a list [r1, r2, …, rk], where ri is the row chosen for the queen in column i and k is the number of queens placed.

Initial state: An empty chessboard with no queens.

Actions: Place one queen in the next column in any row that does not conflict with an already placed queen.

Transition model: Adding a queen to the selected safe square creates the next state.

Goal test: Eight queens have been placed, and no pair shares a row, column, or diagonal.

Path cost: In the basic formulation, each placement can have cost 1; all complete solutions then require eight placements.

Search strategy: Backtracking depth-first search is suitable. Place a queen in a safe row in the next column. If no safe row exists, remove the previous queen and try another row. This prunes invalid states early.

Diagonal check: Queens at (r1, c1) and (r2, c2) share a diagonal if $|r1 - r2| = |c1 - c2|$.

Example solution: [1, 5, 8, 6, 3, 7, 2, 4] gives one valid arrangement when the list records the row for columns 1 through 8.

4. OLA Cab Booking: Problem-Solving Agent and Pseudocode

Agent type: A goal-based problem-solving agent. It receives the pickup and destination, identifies available cab types and routes, and selects an option that reaches the destination while meeting the customer's preferences. A utility-based element can compare fare, estimated travel time, comfort, and availability.

Inputs: Pickup location, destination, preferred cab type (if any), budget/comfort preference, available cabs, estimated fares, and travel times.

BEGIN

Read pickup, destination, and customer preferences

Get available cab types, estimated fares, and travel times

Remove options that are unavailable or do not meet required constraints

IF no suitable cab is available THEN

Display "No suitable cabs available"; ask customer to change preferences

ELSE

Score each option using fare, travel time, comfort, and preference match

Select the highest-scoring suitable cab/route

Display cab type, fare estimate, and arrival/travel time

Ask customer to confirm booking

IF customer confirms THEN book cab and show driver/trip details

ELSE allow customer to choose another option

END IF

END

5. Uniform Cost Search (UCS) for the Delivery Network

The assessment sheet describes a weighted graph with start node S and goal node G, but the graph itself is not visible in the supplied PDF text. Therefore, the exact least-cost route and numerical total cannot be calculated from the provided pages. The correct UCS method is shown below; insert the graph's edges and costs to obtain the specific answer.

Principle: Uniform Cost Search expands the frontier node with the smallest accumulated path cost g(n). It is appropriate when edge costs are non-negative.

1. Put the start node S into a priority queue with cost 0.

2. Set the best-known cost of S to 0; all other nodes initially have infinite cost.

3. Remove the node with the lowest cost from the priority queue.

4. If it is G, return the path and its accumulated cost.

5. For each neighbour v connected by edge cost w, calculate new_cost = cost(current) + w.

6. If new_cost is less than the best-known cost of v, update v's cost and parent, then add/update v in the priority queue.

7. Repeat until G is reached or the queue becomes empty. If the queue becomes empty first, no path exists.

Pseudocode:

UCS(start, goal):

frontier ← priority queue containing (0, start)

cost[start] ← 0; parent[start] ← NIL

while frontier is not empty:

(g, u) ← pop lowest-cost entry

if g is not cost[u]: continue

if u = goal: return reconstruct_path(parent, goal), g

for each edge (u, v) with cost w:

new_cost ← g + w

if new_cost < cost.get(v, infinity):

cost[v] ← new_cost; parent[v] ← u

push (new_cost, v) into frontier

return “No path found”

Note: Please check the original assessment graph for Question 5. It appears to be missing from the supplied PDF pages; the final route and cost depend on that graph.